



Department Software Engineering

Report Project

Subject: Programming Fundamental

Submitted to: Engr. Rehan Ahmed

Submitted by: Sidra Iftikhar

Semester: 1st

Registration no: FA24B1-SE-063

Submission Date: 12 Jan,2025

Title:General Management system(stock)

The program implements a general store management system using c++ language to manage inventory records in a general store. It is specifically designed for the manager/owner of the general store. It allows user to perform various operations such as adding an item, displaying it, searching for an item, calculating total inventory amount, updating the inventory, deleting the expired product from the stock. The system provides validations for security by login function only authorized person can use the system a menu-driven interface.

Code

```
#include "stdafx.h"
#include<iostream>
#include<string>
using namespace std;
struct product
{
    string name;
    int price;
    int quantity;
};
bool login()
{
    string username, password;
    const string validUsername = "Generalstore";
    const string validPassword = "Password";

    cout << "Enter username: ";
    cin >> username;
    cout << "Enter password: ";
    cin >> password;

    if (username == validUsername && password == validPassword)
    {
        return true;
    }
    else
    {
        cout << "Invalid username or password! Access Denied." << endl;
        return false;
    }
}
void addproduct(product items[],int &itemcount)
{
    product item;
    cout<<"Enter product name: ";
    cin>>item.name;
    cout<<"Enter product price: ";
    cin>>item.price;
    cout<<"Enter product quantity: ";
    cin>>item.quantity;
    items[itemcount] = item;
    itemcount++;
    cout<<" Product added successfully !"<<endl;
}
void displayproduct(const product items[],int itemcount)
{

```

```

    cout<<"\n-----Product List-----\n";
    cout<<"Name\tPrice\tQuantity\n";
    for(int i=0;i<itemcount;i++)

cout<<items[i].name<<"\t"<<items[i].price<<"\t"<<items[i].quantity<<endl;
}
void searchproduct(const product items[],int itemcount)
{
    string pname;
    cout<<" Enter product name: ";
    cin>>pname;
    bool found=false;
    for(int i=0;i<itemcount;i++)
    {
        if(items[i].name==pname)
        {
            cout<<"Product found "<<endl;
            cout<<"Name: "<<items[i].name<<endl;
            cout<<"Price: "<<items[i].price<<endl;
            cout<<"Quantity: "<<items[i].quantity<<endl;
            found=true;
            break;
        }
    }
    if(!found)
        cout<<"Product not found "<<endl;
}
void totalInventoryPrice(const product items[], int itemcount)
{
    int total_price = 0;
    for (int i = 0; i < itemcount; i++)
    {
        total_price += items[i].price * items[i].quantity;
    }
    cout << "Total inventory price: " << total_price << endl;
}
void updateInventory(product items[], int &itemcount)
{
    string pname;
    cout << "Enter product name to update: ";
    cin >> pname;

    bool found = false;
    for (int i = 0; i < itemcount; i++)
    {
        if (items[i].name == pname)
        {
            found = true;
            cout << "Product found" << endl;
            cout << "Product Name we want to update: " << items[i].name<<endl;
            cout << "Enter new quantity: ";
            cin >> items[i].quantity;

            cout << "Product updated successfully!" << endl;
            break;
        }
    }

    if (!found)
    {
        cout << "Product not found!" << endl;
    }
}

```

```

    }

    void deleteProduct(product items[], int& itemcount)
    {
        string pname;
        cout << "Enter product name to delete: ";
        cin >> pname;

        bool found = false;
        for (int i = 0; i < itemcount; i++)
        {
            if (items[i].name == pname)
            {
                found = true;

                for (int j = i; j < itemcount - 1; j++)
                {
                    items[j] = items[j + 1];
                }
                itemcount--;
                cout << "Product deleted successfully!" << endl;
                break;
            }
        }

        if (!found)
        {
            cout << "Product not found!" << endl;
        }
    }
}

const int max_item=50;
int _tmain(int argc, _TCHAR* argv[])
{
    product item[max_item];
    int itemcount=0;
    int choice;
    if(!login())
    {
        return 0;
    }
    do
    {
        cout<<"\t General Store Management System \t"<<endl;
        cout<<"1. Add Product "<<endl;
        cout<<"2. Display Product "<<endl;
        cout<<"3. Search Product "<<endl;
        cout<<"4. Total inventory price "<<endl;
        cout<<"5. Update inventory "<<endl;
        cout<<"6. Delete Product"<<endl;
        cout<<"7. End "<<endl;
        cout<<"Enter your choice ";
        cin>>choice;
        switch(choice)
        {
            case 1:
                addproduct(item,itemcount);
                cout<<"-----"<<endl;
                break;
            case 2:
                displayproduct(item,itemcount);
                cout<<"-----"<<endl;
                break;
        }
    }
}

```

```

        case 3:
            searchproduct(item,itemcount);
            cout<<"-----"<<endl;
            break;
        case 4:
            totalInventoryPrice(item,itemcount);
            cout<<"-----"<<endl;
            break;
        case 5:
            updateInventory(item,itemcount);
            cout<<"-----"<<endl;
            break;
        case 6:
            deleteProduct(item,itemcount);
            cout<<"-----"<<endl;
            break;
        case 7:
            cout<<" Program end ";
            break;
        default:
            cout<<" Invalid choice ";
    }
}
    while(choice!=7);
system("pause");
return 0;
}

```

C:\Users\Mega Providers\documents\visual studio 2010\Proje

```

Enter username: Generalstore
Enter password: Password

```

C:\Users\Mega Providers\documents\visual studio 2010\Projects\PF I

```

Enter username: Generalstore
Enter password: Password
    General Store Management System
1. Add Product
2. Display Product
3. Search Product
4. Total inventory price
5. Update inventory
6. Delete Product
7. End
Enter your choice

```

```

Enter your choice 1
Enter product name: oil
Enter product price: 500
Enter product quantity: 2
Product added successfully !
-----
                General Store Management System
1. Add Product
2. Display Product
3. Search Product
4. Total inventory price
5. Update inventory
6. Delete Product
7. End
Enter your choice 1
Enter product name: milk
Enter product price: 200
Enter product quantity: 3
Product added successfully !
-----

```

```

                General Store Management System
1. Add Product
2. Display Product
3. Search Product
4. Total inventory price
5. Update inventory
6. Delete Product
7. End
Enter your choice 2

-----Product List-----
Name    Price   Quantity
oil      500     2
milk     200     3
-----

```

```

                General Store Management System
1. Add Product
2. Display Product
3. Search Product
4. Total inventory price
5. Update inventory
6. Delete Product
7. End
Enter your choice 3
Enter product name: soap
Product not found
-----

```

```

-----
      General Store Management System
1. Add Product
2. Display Product
3. Search Product
4. Total inventory price
5. Update inventory
6. Delete Product
7. End
Enter your choice 4
Total inventory price: 1600
-----

```

```

      General Store Management System
1. Add Product
2. Display Product
3. Search Product
4. Total inventory price
5. Update inventory
6. Delete Product
7. End
Enter your choice 5
Enter product name to update: milk
Product found
Product Name we want to update: milk
Enter new quantity: 4
Product updated successfully!
-----

```

```

      General Store Management System
1. Add Product
2. Display Product
3. Search Product
4. Total inventory price
5. Update inventory
6. Delete Product
7. End
Enter your choice 6
Enter product name to delete: milk
Product deleted successfully!
-----
      General Store Management System
1. Add Product
2. Display Product
3. Search Product
4. Total inventory price
5. Update inventory
6. Delete Product
7. End
Enter your choice 2
-----Product List-----
Name   Price  Quantity
oil     500    2
-----

```

How this system works practically:

This program simulates a **General Store Management System**, which could be practically used to manage inventory for small businesses or retail stores. The features of the program can help in various day-to-day operations, such as:

Managing Products:

- **Adding Products:** Store owners can use the program to add new products to the inventory. For instance, when a new product arrives at the store, they can add it along with its price and quantity.

- **Example:** A shopkeeper could enter products like "Shampoo", "Soap", and "Toothpaste", along with their prices and available stock.

Displaying Product List:

- **Displaying All Products:** The program can display a list of all available products, which can help the store owner review current stock.
- **Example:** The shopkeeper can view a list of all items in the store, including their prices and quantities. This helps in identifying best-selling or low-stock items.

Searching for Products:

- **Searching Products by Name:** This feature enables store managers to search for a specific product in the inventory by name. It helps in quickly finding product details when requested by customers or for restocking purposes.
- **Example:** If a customer asks for a specific brand of soap, the shopkeeper can search for it using the search feature.

Calculating Total Inventory Price:

- **Total Inventory Value:** The program calculates the total value of all products in the store by multiplying the product price with its quantity. This helps store owners understand the total worth of their inventory at any given time.
- **Example:** The shopkeeper may want to know how much capital is tied up in inventory, especially for managing cash flow or calculating taxes.

Updating Inventory:

- **Updating Product Quantities:** This feature allows the user to update the quantity of an existing product. For example, if an item is sold or restocked, the program reflects the change in inventory.
- **Example:** When a customer purchases products, the shopkeeper can adjust the inventory count.

Deleting Products:

- **Removing Products from Inventory:** This function allows the user to remove obsolete or unsold products from the system, keeping the inventory up-to-date and accurate.
- **Example:** If a product is discontinued, or the store needs to remove expired items, the system allows for quick deletion.

Practical Uses:

Security: With the login functionality, only authorized personnel (like the store manager or owner) can access the inventory system, ensuring that no unauthorized changes are made.

Efficiency: Small business owners can use this system to efficiently manage inventory. Rather than manually tracking items, they can perform operations like adding, updating, and deleting products easily with a few commands.

Inventory Control:The program provides a simple way for owners to track and control inventory levels, which helps prevent overstocking or understocking of products.

Financial Management:By tracking the total value of inventory, store owners can have a clear view of their assets. This helps in making informed decisions about investments, restocking, and profit margins.

Customer Satisfaction:The search and display features allow store owners to quickly assist customers by checking product availability or prices, improving customer service.